



Universitat
Oberta
de Catalunya

(HHTMLCSS2) Herramientas HTML y CSS II
PEC 2 - CSS moderno y librerías de componentes.
Bootstrap

Marc Turu Roca

Profesor: Julián Alarte Aleixandre

Fecha de entrega: 12/5/2026

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

1. Introducción	2
2. Configuración inicial	2
3. Guías de estilo y metodologías CSS	2
4. Desarrollo	5
4.1 Idea y diseño	5
4.2 Estructuración de los archivos	7
4.3 Uso de dependencias	8
4.4 Uso de CSS moderno	10
5. Despliegue	12
5.1 Versionado	12
5.2 Publicación a internet	12
6. Recursos	13
7. Anexos	14

1. Introducción

Esta práctica ha consistido en el desarrollo de un sitio web formado por 4 páginas, partiendo de [UOC Boilerplate](#) y aplicando conceptos propios de CSS moderno y librerías de componentes como Bootstrap, entre otros.

El repo de **GitHub** es: <https://github.com/marcturu/ac-materia>

El link para acceder a la **web** es: <https://ac-materia.netlify.app/>

2. Configuración inicial

En esta primera parte, se hizo un `git clone`

<https://github.com/uoc-advanced-html-css/uoc-boilerplate.git> para poder disponer de la plantilla inicial sobre la cual trabajar y se enlazó al nuevo repositorio creado (<https://github.com/marcturu/ac-materia>) ejecutando:

```
git remote add origin https://github.com/marcturu/ac-materia
```

```
git branch -M main
```

 para cambiar la rama principal.

```
git push -u origin main
```

 para añadir los cambios al repositorio de GitHub.

Seguidamente, se hizo un `npm install --save @fortawesome/fontawesome-free` e `@import "npm:@fortawesome/fontawesome-free/css/all.css";` en `src/assets/styles/_dependencies.scss` para poder usar los iconos de Font Awesome. Al igual que `npm install @popperjs/core` `bootstrap --save` e `@import "bootstrap/scss/bootstrap";` en `src/assets/styles/_dependencies.scss` para usar Bootstrap e `import * as bootstrap from bootstrap;` en `src/assets/scripts/main.js` para importar también el JavaScript de sus funcionalidades.

3. Guías de estilo y metodologías CSS

Después de estudiar las diferentes guías de estilo, se decidió adoptar [CSS Guidelines](#) como referencia principal, acompañada de las metodologías ITCSS, OCSS y BEM. La elección no respondió únicamente a preferencias personales, sino a la necesidad de construir una arquitectura CSS mantenible, escalable y coherente con los requisitos técnicos y metodológicos planteados en esta práctica, aparte de resultar especialmente adecuada al permitir abordar simultáneamente la organización global, la reutilización de estilos y la nomenclatura de componentes.

- **ITCSS** como arquitectura global del proyecto.

Su principal aportación fue organizar los estilos desde las capas más genéricas y de menor especificidad hasta las más concretas y específicas. Este enfoque encajaba

especialmente bien con el uso de SASS mediante **parciales e importaciones**, la coexistencia entre **estilos propios** y **Bootstrap** y la utilización de **capas de cascada**.

La estructura de las carpetas se puede observar en la sección “*Estructura del proyecto*” en el [README.md de GitHub](#).

- **OCSS** para reutilización y separación de responsabilidades.

Su principal aportación fue reutilizar patrones visuales y separar estructura de apariencia. Debido a que en el enunciado se planteaba desarrollar múltiples páginas con *layouts responsive*, OCSS permitió **evitar duplicar** código, mejorar la **consistencia visual** y **reutilizar estructuras** comunes en las páginas sin repetir reglas.

Como ejemplo, fue útil para representar la separación visual de las disposiciones de los contenedores que se podía observar en los *wireframes*.

- **BEM** para nomenclatura y mantenibilidad.

Su principal aportación fue **evitar colisiones** entre numerosos componentes al mantener una nomenclatura clara, **comprender rápidamente** la función de cada clase al no tener selectores excesivamente anidados y mantener el **código escalable**. Además, encajaba muy bien con SASS nesting moderado y con la arquitectura *ITCSS*, ya que cada componente podía mantenerse encapsulado dentro de su propio parcial.

Como ejemplo, fue útil para personalizar los (BE)*Modifiers* de `hero__item--` que permitieron personalizar los contenedores de la *Grid* de la portada.

Esta combinación representaba una aproximación cercana a entornos reales de desarrollo *frontend*, donde es habitual combinar distintas metodologías según el problema que resuelven para construir un sistema de estilos robusto.

También se instaló la herramienta de análisis estático y validación de estilos **Stylelint** (después de consultar la compatibilidad de versiones con `npm info <dependencia> peerDependencies`) con el siguiente comando: `npm install --save-dev1 stylelint@17 stylelint-scss@7 stylelint-config-recommended-scss@17 stylelint-config-prettier-scss` y se añadió la línea `"stylelint": "stylelint src**/*.scss"` en `package.json` para usarla al construir la *build*.

- **stylelint** como núcleo principal de la herramienta.

¹ Se añadió `--save-dev` para que las dependencias apareciesen en el *devDependencies* y se usaran en el entorno de desarrollo.

Analiza archivos y detecta errores sintácticos, malas prácticas, inconsistencias, duplicaciones y problemas potenciales de mantenibilidad.

- **stylelint-scss** como plugin específico para SASS.
Añade soporte para variables SASS, *nesting*, *mixins*, funciones, placeholders e interpolaciones. Además, habita reglas propias como `"scss/dollar-variable-pattern"` y `"scss/at-rule-no-unknown"`.
- **stylelint-config-recommended-scss** como configuración base recomendada para proyectos SCSS.
Incluye reglas de validación, detección de errores comunes y compatibilidad con SASS moderno y sirve como punto de partida para evitar definiciones manuales.
- **stylelint-config-prettier-scss** como desactivador de reglas de Stylelint que podrían entrar en conflicto con *Prettier*.

Además, se configuró el archivo `.stylelint.rc` para personalizar reglas:

`"color-hex-length": "short"` para forzar hexadecimales abreviados cuando sea posible.

`"comment-no-empty": true` para evitar comentarios vacíos innecesarios.

`"block-no-empty": true` para evitar bloques de estilos vacíos.

`"color-no-invalid-hex": true` para detectar colores hexadecimales inválidos.

`"declaration-block-no-duplicate-properties": true` para evitar propiedades duplicadas dentro del mismo bloque.

`"declaration-block-no-shorthand-property-overrides": true` para detectar conflictos entre *shorthand* y *longhand*.

`"no-duplicate-selectors": true` para impedir definir varias veces el mismo selector en un mismo contexto.

`"no-descending-specificity": true` para evitar problemas de cascada donde selectores menos específicos sobreescriben otros más específicos en orden incorrecto.

`"selector-attr-id": 0` para prohibir selectores por ID y reforzar la coherencia BEM.

`"at-rule-no-unknown": null` i `"scss/at-rule-no-unknown": true` para desactivar la validación estándar CSS y sustituirla por la versión *SCSS-aware*.

`"scss/dollar-variable-pattern": "^[a-z][a-z0-9]*(-[a-z0-9]+)*$"` para obligar a que las variables sigan *kebab-case* y evitar estilos inconsistentes.

`"scss/percent-placeholder-pattern": "%[a-z0-9-]+%"` para controlar la nomenclatura de placeholders SASS.

`"selector-class-pattern": "^[a-z]([a-z0-9-]+)?(__([a-z0-9-]+-?)+)?(--([a-z0-9-]+-?)+){0,2}$"` para validar automáticamente que las clases sigan el patrón BEM adoptado en el proyecto.

Permite estructuras como `.card`, `.card__body`, `.card__body-active`.

4. Desarrollo

4.1 Idea y diseño

Para la página web, debido a que la idea era elaborar un portal sobre unas jornadas artesanales, se quiso dar un tono cercano a la vez que creativo e inmersivo. Es por eso que se definieron las fuentes *Lethal Gothic* (para títulos) y *DM Sans* (para *body*) para compensar el carácter expresivo de la primera con la limpieza de la segunda. Aparte, la paleta de colores declarada en las `_variables.scss`, consistía de los colores básicos en hexadecimal como *fallback* acompañados por los de tipo **OKLCH** para dotar de mayor consistencia perceptual entre tonalidades, mejor control de contraste y luminosidad, transiciones cromáticas más naturales y una representación del color más cercana a la percepción humana. La implementación se realizó mediante consultas `@supports`, permitiendo utilizar automáticamente los colores OKLCH (y **display-p3**) únicamente en navegadores compatibles:

```
color: v.$color-primary-fallback;
@supports (color: oklch(62% 0.15 50)) {
  color: v.$color-primary;
}
```



Fig. 1 Logo de la asociación.

Un lugar muy importante donde se usó la consulta `@supports` fue en la construcción del poster de la portada, la cual se construyó con **CSS Grid**. A pesar de ello, debido a que pueden haber navegadores no compatibles con su uso, se añadió código a modo de *fallback* para disponer siempre de un *layout* coherente²:

```
&__grid {
  display: flex;
  ...
  @supports (display: grid) {
    display: grid;
    ...
  }
}
```

² Se aplicó tanto en la cuadrícula como en los elementos de las celdas.

Por otro lado, como los *wireframes* iniciales eran en formato ordenador, aparte de mantener los *mixins* con los *breaking points* ajustados a *min-width*, se incluyeron otros con ***max-width*** para poder seguir un diseño *desktop-first* y, partiendo de los *mockups*, adaptarlos a tamaños más reducidos. Así, con estas dos funciones:

```
@mixin respond-to($bp) { ... } @mixin respond-to-max($bp) { ... }
```

se podían adaptar componentes dependiendo de los *breaking points* establecidos³, como por ejemplo con una imagen de la página Blog que se reducía dinámicamente:

```
height: 600px;
@include m.respond-to-max(md) {
    height: 300px;
}
```

Sin embargo, a pesar de que las *media queries* son muy útiles para conseguir diseños adaptables, también se incorporaron *container queries* (**@container**) para adaptar componentes según el tamaño real de su contenedor y no únicamente del *viewport*, siguiendo una aproximación más modular y moderna.

Por ejemplo, las tarjetas de los ponentes se ajustaban dinámicamente:

```
@container speaker-card (max-width: 320px) {
    height: 380px;
}
```

después de declarar los siguientes valores en su contenedor padre:

```
container-type: inline-size;
container-name: speaker-card;
```

Además, se utilizaron funciones fluidas como **@clamp()** y nuevas unidades relativas modernas como **cqw**, **dvh**,... para conseguir escalados tipográficos y espaciales más naturales entre resoluciones intermedias, evitando saltos bruscos entre *breakpoints*.

```
@container speaker-card (max-width: 500px) {
    .speakers-card__name {
        font-size: clamp(2rem, 5cqw, 2.5rem);
    }
}
```

Así que, la combinación de *CSS Grid*, *Flexbox*, *Bootstrap*, *media queries*, *container queries* y unidades fluidas *responsive*, permitieron diseñar una interfaz completamente *responsive* y adaptable a dispositivos móviles, tablets y pantallas de escritorio manteniendo coherencia visual y buena UX en todos los formatos.

³ Los *breaking points* siguieron la estructura de Bootstrap: 576px, 768px, 992px, 1200px, 1400px.

4.2 Estructuración de los archivos

```
src/
├─ index.html
├─ blog.html
├─ speakers.html
├─ contact.html
├─ views/
│   ├─ partials/           # Parciales PostHTML
│   │   ├─ header.html
│   │   └─ footer.html
│   ├─ home/
│   │   └─ hero.html
│   ├─ speakers/
│   │   └─ cards.html
│   ├─ blog/
│   │   ├─ hero.html
│   │   ├─ intro.html
│   │   ├─ list.html
│   │   └─ final.html
│   └─ contact/
│       ├─ hero.html
│       ├─ section.html
│       ├─ cards.html
│       ├─ form.html
│       └─ map.html
├─ assets/
│   ├─ fonts/             # Fuentes autoalojadas (League Gothic, DM Sans)
│   ├─ images/            # Imágenes optimizadas en WebP
│   ├─ scripts/
│   │   ├─ main.js
│   │   └─ modules/      # Módulos JS (aos, header)
│   └─ styles/
│       ├─ main.scss
│       ├─ layers.scss
│       ├─ _dependencies.scss
│       ├─ settings/      # Variables, fuentes, bootstrap overrides
│       ├─ tools/         # Funciones, mixins
│       ├─ generic/       # Reset
│       ├─ elements/      # Estilos base
│       ├─ objects/       # Objetos OOCSS
│       ├─ components/
│       │   ├─ partials/
│       │   ├─ home/
│       │   ├─ speakers/
│       │   ├─ blog/
│       │   └─ contact/
│       └─ utilities/     # Clases de utilidad
```

Fig. 2 - Estructura de archivos del proyecto disponible en el [README.md de GitHub](#).

Se separaron las diferentes secciones y sus vistas, los diferentes recursos **assets/**, las diferentes imágenes **images/**, los diferentes **scripts** **modules/** y los diferentes estilos ITCCS **styles/**.

También se optó por mejorar la resolución de las imágenes **.webp** con [Squoosh](#).

4.3 Uso de dependencias

En cuanto a las dependencias externas, primeramente se instaló **FontAwesome** (^2.7.0) para utilizar sus iconos en 3/4 páginas de la siguiente manera:

```
<i class="fa-solid fa-calendar" aria-hidden="true"></i>
```

También se instaló **Bootstrap** (^5.3.8), el cual, después de importarlo en JS (`import * as bootstrap from 'bootstrap'`) y SCSS, se creó un `_bootstrap-overrides.scss` donde se sobrescribieron distintas variables de Bootstrap, entre ellas:

Modify map, a partir del cual Bootstrap genera clases básicas (`.btn-primary`, p.e.):

```
$theme-colors: (
  "primary": $color-primary-fallback,
  "secondary": $color-secondary-fallback,
  "light": $color-light-fallback,
  "dark": $color-dark-fallback
);
```

Atributos generales:

```
$font-family-base: v.$font-secondary;
$font-family-headings: v.$font-primary;
```

Atributos de componentes Bootstrap:

```
$link-color: v.$color-light-fallback;
$link-hover-color: v.$color-primary-fallback;
```

```
$btn-border-radius: v.$spacing-xs;
```

```
$navbar-nav-link-padding-x: v.$spacing-sm;
$navbar-light-color: v.$color-secondary-fallback;
$navbar-light-hover-color: v.$color-primary-fallback;
$navbar-light-active-color: v.$color-primary-fallback;
```

```
$dropdown-link-color: v.$color-secondary-fallback;
$dropdown-link-hover-color: v.$color-primary-fallback;
```

```
$card-title-color: v.$color-secondary-fallback;
$card-subtitle-color: v.$color-primary-fallback;
$card-border-color: v.$color-border-fallback;
$card-border-radius: v.$spacing-xs;
```

```
$input-border-radius: v.$spacing-xs;
$input-line-height: 1;
$form-select-padding-y: v.$spacing-xs;
$form-select-border-radius: v.$spacing-xs;
```

Estos cambios permitieron dotar a los componentes de Bootstrap de estilos personalizados que seguían un mismo flujo de diseño en torno a la paleta de

colores, estilos y layout general, permitiendo regirse a los *wireframes* y llevar a cabo la idea de diseño inicial de una manera más fiel.

Como se puede apreciar en las personalizaciones, los componentes Bootstrap utilizados más destacados fueron **Navbar** (para el *header*), **Dropdown** (para el elemento ponentes del *header*), **Card** (para los ponentes), **Form** (para el formulario de contacto) y **Links** y **Botones** usados como complemento en distintas páginas de la web.

Por otro lado, se dió uso a *utility classes* y *layout responsive* para complementar componentes propios (o de Bootstrap), como por ejemplo `mb-*` para dar margen, `ms-auto` para definir *margin-inline-start*, `navbar-expand-lg` para expandir la *navbar* horizontalmente al colapsar en `< lg`, entre otros.

Como última dependencia, se instaló la librería externa **AOS** (^2.3.4) con el objetivo de añadir pequeñas animaciones de entrada asociadas al *scroll* y reforzar el carácter visual e inmersivo del sitio web.

La integración de AOS resultó especialmente adecuada porque no requirió de *JavaScript* complejo, funcionó mediante atributos *HTML* declarativos y encajó con la idea de diseño. Es por eso que se decidió incorporar estas animaciones en el poster de la portada⁴ y dotar a sus elementos de dinamismo visual al entrar al *viewport*:

```
data-aos="fade-left" data-aos-delay="100">
```

Posteriormente, la librería se inicializaba desde JavaScript mediante:

```
import AOS from 'aos';
import 'aos/dist/aos.css';

export function initAOS() {
  AOS.init({
    duration: 550,
    easing: 'ease-out',
    once: false,
    offset: 60,
  });

  window.addEventListener('load', () => AOS.refresh());
  window.addEventListener('resize', () => AOS.refresh());
}
```

donde se declaraban valores como el estilo de las animaciones, su duración, cuando se activaban y cómo.

⁴ No se añadió a más páginas para no sobrecargar visualmente la interfaz ni perjudicar el rendimiento

4.4 Uso de CSS moderno

Uno de los principales objetivos del proyecto fue aplicar de forma práctica varias de las características modernas de estilos. Para ello se incorporaron técnicas actuales relacionadas con control avanzado de cascada, diseño responsivo basado en componentes, *pseudoclasas* funcionales, consultas de soporte y *progressive enhancement*.

Capas de cascada (@layer)

Se utilizaron capas de cascada para controlar explícitamente la prioridad y el orden de carga de los estilos, reduciendo conflictos de especificidad y facilitando la coexistencia entre estilos. El orden para la práctica fue:

```
@layer reset, base, objects, components, article-base, article-components,  
contact-base, contact-components, utilities;
```

Este enfoque permitió organizar mejor la arquitectura CSS, reforzar la filosofía ITCSS aplicada en el proyecto y mantener perceptible la cascada, ya que, aparte de apreciarse en el código, se podía observar fácilmente qué *layer* cargaba cada estilo con la inspección de la página en las *Dev Tools*.

Container Queries (@container)

Se incorporaron *container queries* tradicionales para construir componentes responsive independientes del tamaño del *viewport*. A diferencia de las *media queries* tradicionales, estas permitieron adaptar componentes según el tamaño real del contenedor donde se encontraban, favoreciendo una arquitectura mucho más modular y reutilizable.

```
container-type: inline-size;  
container-name: hero-title-block;
```

Pseudoclasas funcionales modernas (:has(), :is() y :where())

La pseudoclase `:has()` se utilizó para aplicar estilos condicionales según el estado o contenido interno de determinados elementos, algo que tradicionalmente requería JavaScript.

Por ejemplo, después de asegurar un *fallback* y comprobar la posibilidad del uso de la pseudoclase con `@supports selector(has(*))` (debido a que no está *Widely available* como `:is()` y `:where()`), se añadió espacio visual a una sección si esta contenía un *blockquote*:

```
.article__reflection-body:has(.article__blockquote) { gap: v.$spacing-sm; }
```

También se utilizaron `:is()` y `:where()` para simplificar selectores complejos y reducir repetición de código. Estas pseudoclases permitían agrupar selectores manteniendo una especificidad más controlada, favoreciendo una arquitectura CSS más mantenible. Por ejemplo:

```
:is(h2, h3) {  
  margin-bottom: v.$spacing-md;  
}  
  
:where(:not(.article__section--fullwidth)) {  
  padding-inline: 0;  
}
```

El uso de `:where()` resultó especialmente útil debido a su especificidad nula, encajando muy bien con la filosofía ITCSS y el control de la cascada.

Consultas de soporte (@supports)

Se utilizaron consultas de soporte para aplicar *progressive enhancement* y garantizar compatibilidad con navegadores que no soportasen determinadas características modernas.

Uno de los casos más relevantes fue la construcción de la portada mediante CSS Grid. Aunque el layout principal se desarrolló utilizando Grid, se añadió una alternativa basada en Flexbox como *fallback*:

```
&__grid {  
  display: flex;  
  ...  
  @supports (display: grid) {  
    display: grid;  
    ...  
  }  
}
```

De este modo, navegadores modernos utilizaban el layout avanzado con Grid, mientras que navegadores sin soporte seguían mostrando una estructura funcional y coherente.⁵

Las consultas `@supports` también se utilizaron para colores modernos OKLCH y `display-p3`, efectos visuales avanzados y determinadas propiedades experimentales.

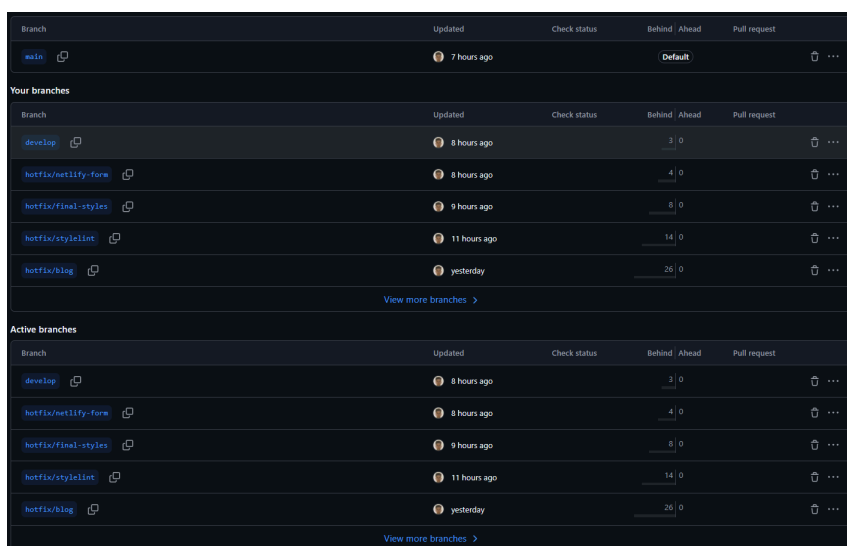
⁵ Se puede observar la diferencia de `display` en los anexos.

5. Despliegue

5.1 Versionado

En el repositorio del código, se mantuvo la rama **main** para los “lanzamientos” de las versiones finales, se creó una rama **develop** para poder juntar las diferentes **features/xx-nombre-feature** y se reservaron las ramas **hotfix/nombre-hotfix** para resolver problemas que surgieran en el código a lo largo del proyecto.

Para publicar los cambios se hacía `$ git add <files> $ git commit -m “<message>” $ git push <branch>`, y para compilar para producción (**/dist**), el comando del `package.json` “build”: “npm-run-all clean stylelint parcel:build”: `$ npm run build`.



Branch	Updated	Check status	Behind	Ahead	Pull request
main	7 hours ago			Default	
Your branches					
develop	8 hours ago		3	0	
hotfix/netlify-form	8 hours ago		4	0	
hotfix/final-styles	9 hours ago		8	0	
hotfix/stylelint	11 hours ago		14	0	
hotfix/blog	yesterday		26	0	
View more branches >					
Active branches					
develop	8 hours ago		3	0	
hotfix/netlify-form	8 hours ago		4	0	
hotfix/final-styles	9 hours ago		8	0	
hotfix/stylelint	11 hours ago		14	0	
hotfix/blog	yesterday		26	0	
View more branches >					

Fig. 3 - Ramas de GitHub

5.2 Publicación a internet

Para poder publicar la web en internet y que estuviera disponible, se tuvo que crear y configurar un nuevo proyecto en **Netlify**. En él, después de especificar el nombre, se estableció la rama **dist/** (la cual era la carpeta que contenía el código optimizado con *Parcel* y listo para producción) como el directorio de publicación y **npm run build** como el comando de construcción. De esta manera, cada vez que se realizaba un cambio en el repositorio en la rama **main**, se creaba un **deploy** con referencia al historial del último **commit** de esta.

Además, en Netlify se podían consultar los datos recolectados del **form** creado con `data-netlify="true"` en el apartado **Forms**, el cual ofrecía información de las respuestas.⁶

⁶ Se puede observar esta información en los anexos.

6. Recursos

- [Can I use?](#)

Información sobre compatibilidades con navegadores de recursos CSS.

- [Unsplash](#)

Imágenes de portada, ponentes y blog obtenidas de Unsplash bajo licencia gratuita para uso comercial y personal.⁷

- [Squoosh](#)

Compresor y optimizador de imágenes directamente desde el navegador y con variedad de formatos.

- [Canva](#)

Herramienta online de diseño gráfico.

- [AOS — Animate On Scroll](#)

Librería utilizada para las animaciones al hacer scroll.

- [Font Awesome](#)

Librería de iconos utilizada en todo el proyecto.

- [Bootstrap](#)

Documentación sobre la herramienta Bootstrap y sus componentes.

- [Stylelint](#)

Documentación del *linter* CSS.

- [Open Street Map](#)

Mapa *open-source* de cartografía libre.

⁷ Se puede observar dicha licencia en los anexos.

7. Annexos

1. Aplicación web. Asociación Cultural Materia - Jornadas Artesanales (Netlify)

<https://ac-materia.netlify.app/>

2. Vídeo explicación

<https://drive.google.com/file/d/1coeH08jkauLuaykaHTYhFzpioiWx2eHD/view?usp=sharing>

(durante los 10 primeros segundos no se ve del todo bien, pero después se arregla. Solamente ocurre cuando se accede a través del **link compartido**, ya que tanto el vídeo original descargado como el archivo en el Drive, se ven bien)

3. Repositorio con el código (GitHub)

<https://github.com/marcturu/ac-materia>

4. Poster con navegador que soporta CSS Grid (1) y que no (2):



1



2

5. Respuesta de ejemplo en Netlify Forms:

Submissions from contact

Last submission at 4:38 PM (a few seconds ago).

Extra spam prevention enabled via honeypot field.

[Learn more about form handling in the docs](#)

[Download as CSV](#)
[Delete form](#)

Verified submissions

☐ Select all on page
 [Delete submission](#)
[Mark as spam](#)
[Expand all](#)

☐

Mensaje de ejemplo.

Nombre y apellidos (obligatorio) Test Tes Te

Correo electrónico (obligatorio) test@example.com

Motivo de la consulta (obligatorio) taller

Mensaje (obligatorio) Mensaje de ejemplo.

He leído y acepto la política de privacidad on

Received today at 4:38 PM from 79.158.237.150

4:38 PM (a few seconds ago)

6. Licencia Unsplash:

Licencia

Las imágenes de Unsplash se utilizan libremente. Nuestra licencia lo refleja.

- ✓ Todas las imágenes se pueden **descargar** y utilizar de **forma gratuita**
- ✓ **Fines comerciales y no comerciales**
- ✓ **No se necesita permiso** (¡aunque se agradece la atribución!)

Qué no está permitido 🚫

- ⊘ Las imágenes no se pueden **vender** sin modificaciones significativas.
- ⊘ La recopilación de imágenes de Unsplash para replicar un servicio similar o contrapuesto.

Archivos

Unsplash te concede una licencia de derechos de autor irrevocable, no exclusiva y mundial para descargar, copiar, modificar, distribuir, realizar y utilizar imágenes de Unsplash de forma gratuita, incluso con fines comerciales, sin el permiso o la atribución del fotógrafo o Unsplash. Esta licencia no incluye el derecho a compilar imágenes de Unsplash para replicar un servicio similar o contrapuesto.